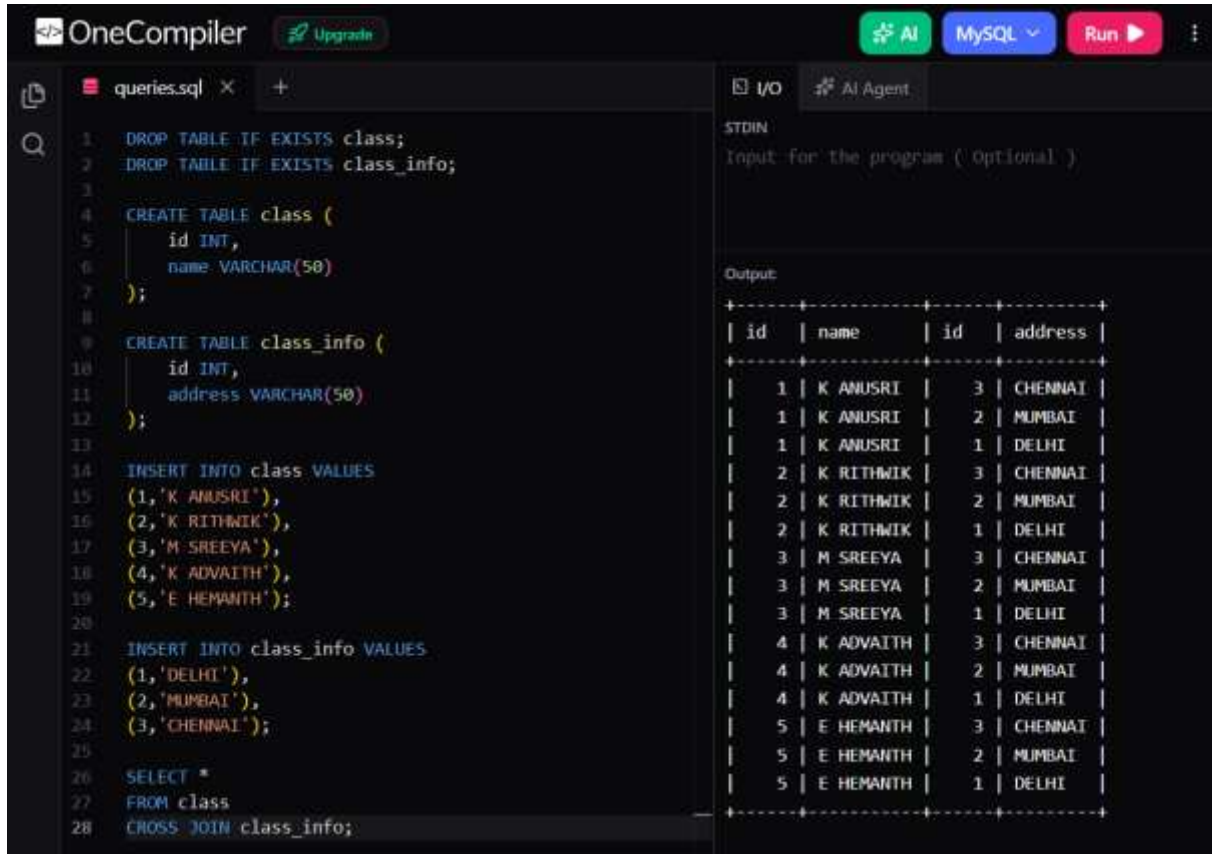


SQL JOINS & SET OPERATIONS

PART 1 — CROSS JOIN

1. Cross Join — Pair Every Student with Every City



The screenshot shows the OneCompiler MySQL interface. The SQL editor on the left contains the following code:

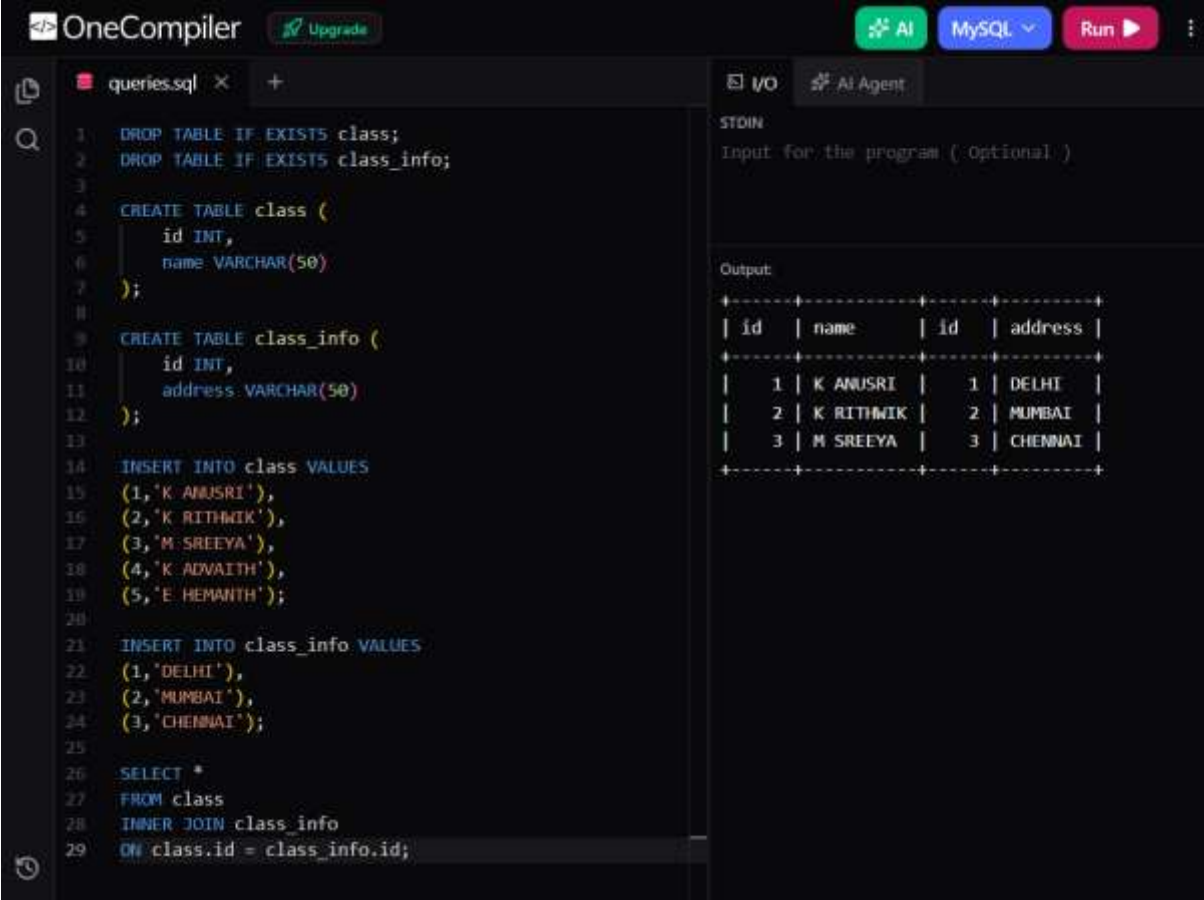
```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10    id INT,
11    address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1, 'K ANUSRI'),
16 (2, 'K RITHWIK'),
17 (3, 'M SREEYA'),
18 (4, 'K ADVAITH'),
19 (5, 'E HEMANTH');
20
21 INSERT INTO class_info VALUES
22 (1, 'DELHI'),
23 (2, 'MUMBAI'),
24 (3, 'CHENNAI');
25
26 SELECT *
27 FROM class
28 CROSS JOIN class_info;
```

The output on the right shows a table with 15 rows, representing all possible combinations of students and cities:

id	name	id	address
1	K ANUSRI	3	CHENNAI
1	K ANUSRI	2	MUMBAI
1	K ANUSRI	1	DELHI
2	K RITHWIK	3	CHENNAI
2	K RITHWIK	2	MUMBAI
2	K RITHWIK	1	DELHI
3	M SREEYA	3	CHENNAI
3	M SREEYA	2	MUMBAI
3	M SREEYA	1	DELHI
4	K ADVAITH	3	CHENNAI
4	K ADVAITH	2	MUMBAI
4	K ADVAITH	1	DELHI
5	E HEMANTH	3	CHENNAI
5	E HEMANTH	2	MUMBAI
5	E HEMANTH	1	DELHI

PART 2 — INNER JOIN

1. Inner Join - Students with Available Addresses



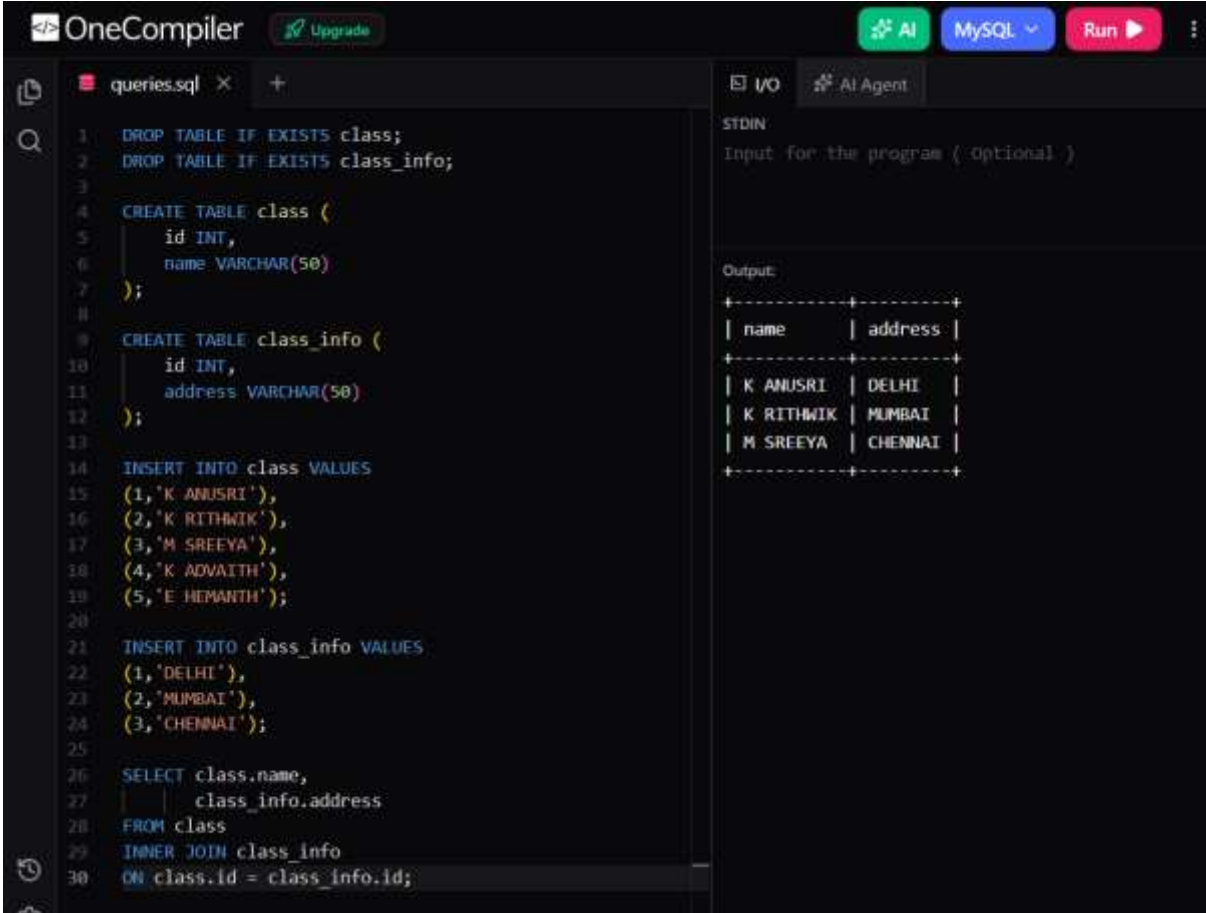
The screenshot shows the OneCompiler IDE with a MySQL database. The code defines two tables: 'class' and 'class_info'. The 'class' table has columns 'id' (INT) and 'name' (VARCHAR(50)). The 'class_info' table has columns 'id' (INT) and 'address' (VARCHAR(50)). Data is inserted into both tables. Finally, an inner join is performed on the 'id' columns of both tables.

```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10    id INT,
11    address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1, 'K ANUSRI'),
16 (2, 'K RITHMIK'),
17 (3, 'M. SREEYA'),
18 (4, 'K ADVAITH'),
19 (5, 'E HEMANITH');
20
21 INSERT INTO class_info VALUES
22 (1, 'DELHI'),
23 (2, 'MUMBAI'),
24 (3, 'CHENNAI');
25
26 SELECT *
27 FROM class
28 INNER JOIN class_info
29 ON class.id = class_info.id;
```

Output:

id	name	id	address
1	K ANUSRI	1	DELHI
2	K RITHMIK	2	MUMBAI
3	M SREEYA	3	CHENNAI

2. Inner Join — Display Student Names and Addresses



The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

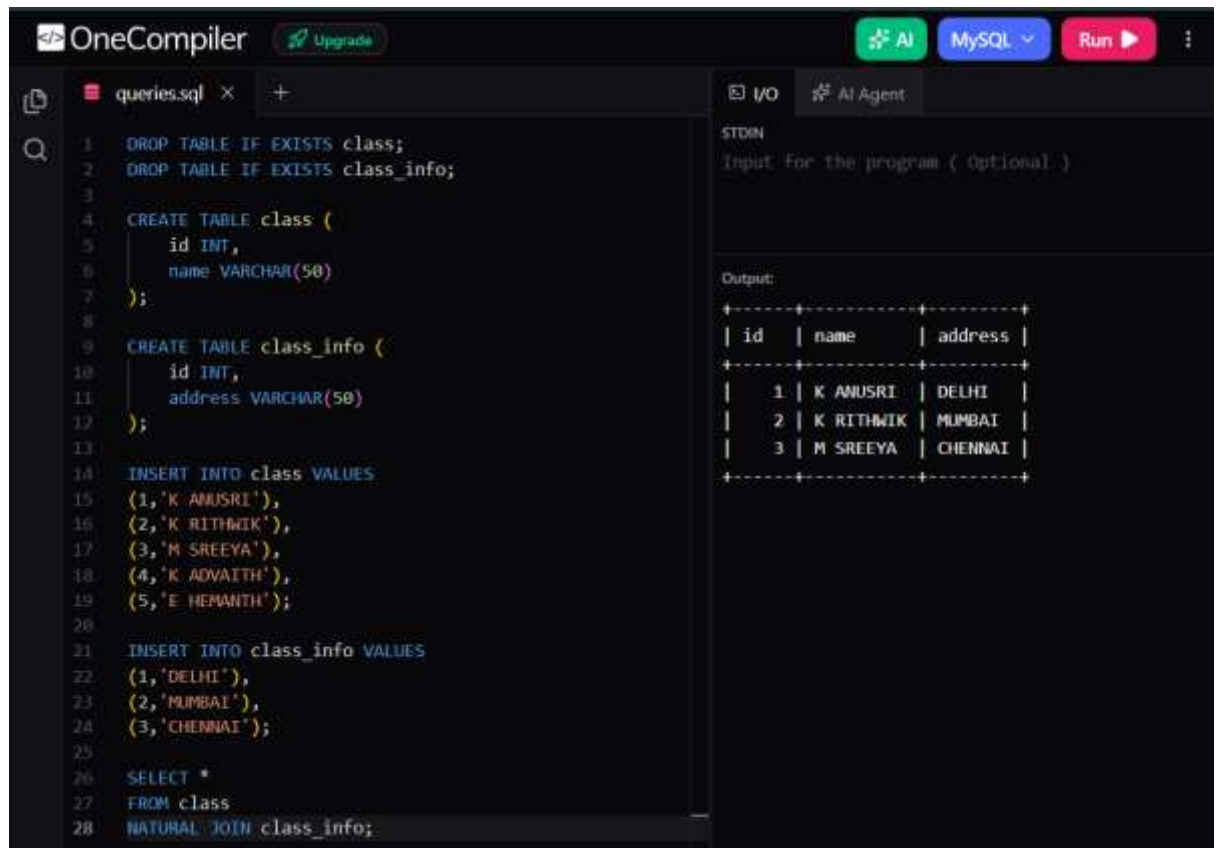
```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10    id INT,
11    address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1, 'K ANUSRI'),
16 (2, 'K RITHWIK'),
17 (3, 'M SREEYA'),
18 (4, 'K ADVAITH'),
19 (5, 'E HEMANTH');
20
21 INSERT INTO class_info VALUES
22 (1, 'DELHI'),
23 (2, 'MUMBAI'),
24 (3, 'CHENNAI');
25
26 SELECT class.name,
27        class_info.address
28 FROM class
29 INNER JOIN class_info
30 ON class.id = class_info.id;
```

The right pane shows the output of the query, which is a table with two columns: name and address. The output is as follows:

name	address
K ANUSRI	DELHI
K RITHWIK	MUMBAI
M SREEYA	CHENNAI

PART 3 — NATURAL JOIN

1. Natural Join — Automatically Match Common ID



The screenshot shows the OneCompiler MySQL interface. The left pane contains a file named 'queries.sql' with the following SQL code:

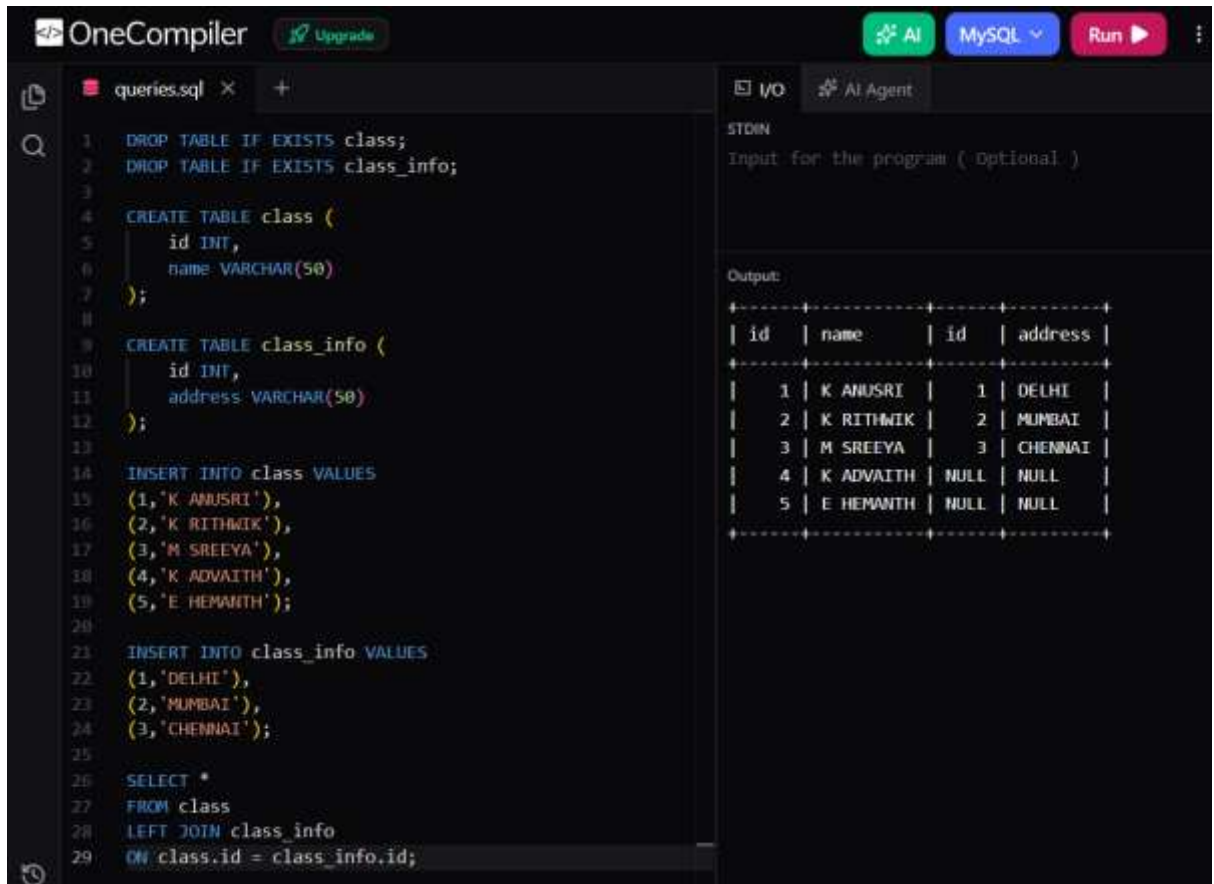
```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10    id INT,
11    address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1, 'K ANUSRI'),
16 (2, 'K RITHVIK'),
17 (3, 'M SREEYA'),
18 (4, 'K ADVAITH'),
19 (5, 'E HEMANTH');
20
21 INSERT INTO class_info VALUES
22 (1, 'DELHI'),
23 (2, 'MUMBAI'),
24 (3, 'CHENNAI');
25
26 SELECT *
27 FROM class
28 NATURAL JOIN class_info;
```

The right pane shows the output of the query. It includes a 'STDIN' section for input and an 'Output:' section displaying the result of the NATURAL JOIN query. The output is a table with 3 rows and 3 columns: id, name, and address.

id	name	address
1	K ANUSRI	DELHI
2	K RITHVIK	MUMBAI
3	M SREEYA	CHENNAI

PART 4 — LEFT OUTER JOIN

1. Left Join — Show All Students



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script in a file named `queries.sql`. The script creates two tables, `class` and `class_info`, inserts data into them, and then performs a left join to display all students.

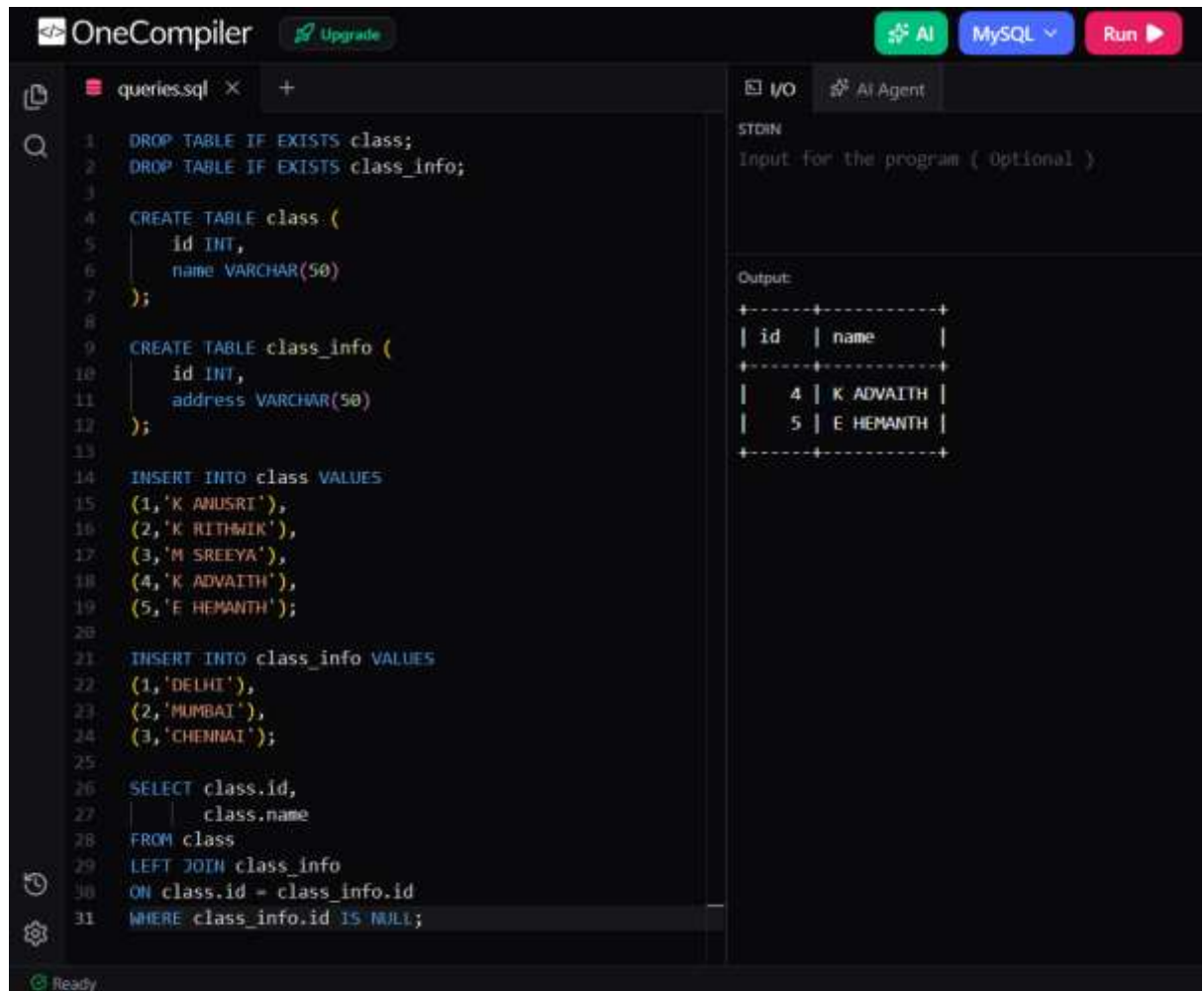
```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10    id INT,
11    address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1, 'K ANUSRI'),
16 (2, 'K RITHWIK'),
17 (3, 'M SREEYA'),
18 (4, 'K ADVAITH'),
19 (5, 'E HEMANTH');
20
21 INSERT INTO class_info VALUES
22 (1, 'DELHI'),
23 (2, 'MUMBAI'),
24 (3, 'CHENNAI');
25
26 SELECT *
27 FROM class
28 LEFT JOIN class_info
29 ON class.id = class_info.id;
```

The right pane shows the output of the query. It includes a section for STDIN (Input for the program (Optional)) and an Output section displaying the result of the left join in a table format.

Output:

id	name	id	address
1	K ANUSRI	1	DELHI
2	K RITHWIK	2	MUMBAI
3	M SREEYA	3	CHENNAI
4	K ADVAITH	NULL	NULL
5	E HEMANTH	NULL	NULL

2. Left Join — Find Students Without Addresses



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL query in a file named 'queries.sql'. The query is as follows:

```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10    id INT,
11    address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1, 'K ANUSRI'),
16 (2, 'K RITHVIK'),
17 (3, 'M SREEYA'),
18 (4, 'K ADVAITH'),
19 (5, 'E HEMANTH');
20
21 INSERT INTO class_info VALUES
22 (1, 'DELHI'),
23 (2, 'MUMBAI'),
24 (3, 'CHENNAI');
25
26 SELECT class.id,
27        class.name
28 FROM class
29 LEFT JOIN class_info
30 ON class.id = class_info.id
31 WHERE class_info.id IS NULL;
```

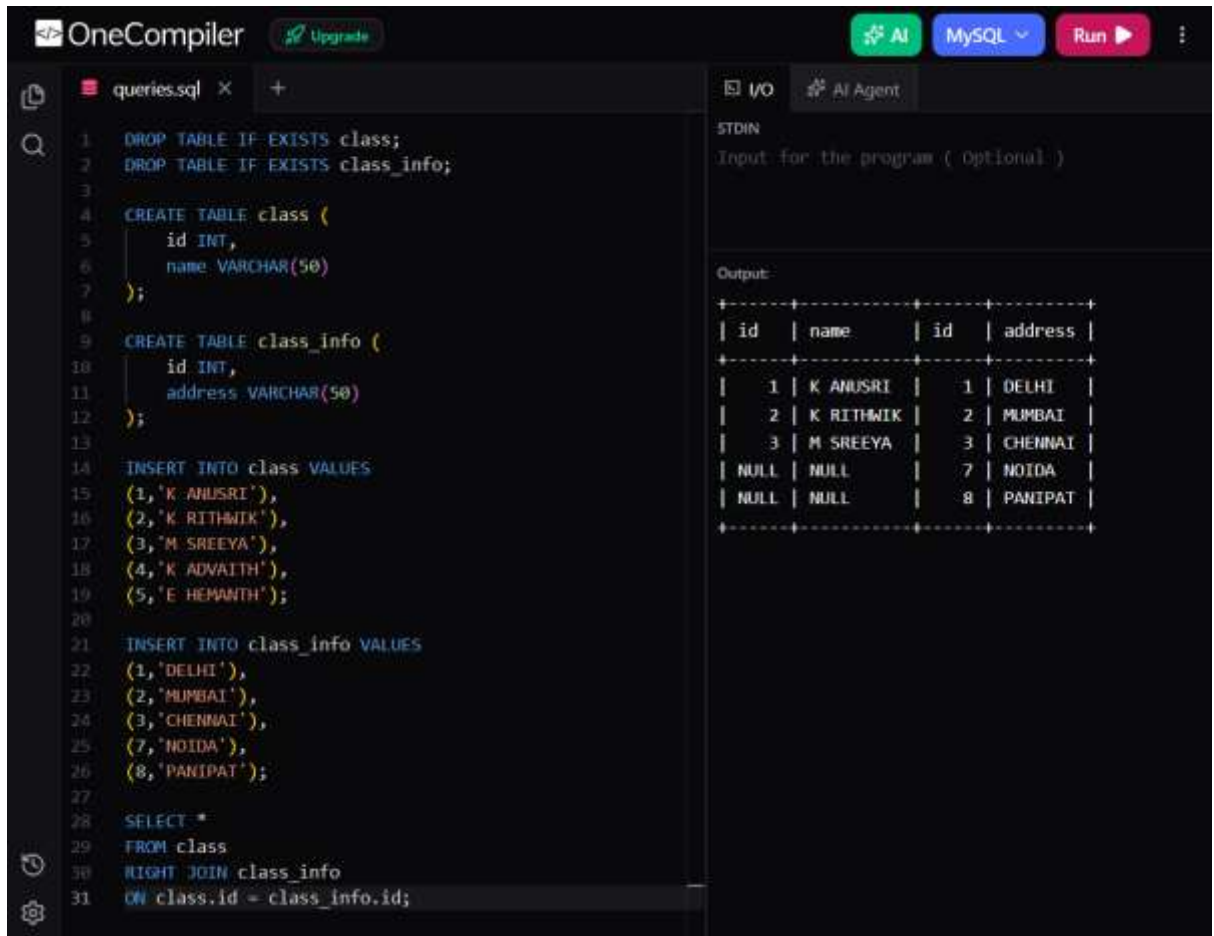
The right pane shows the output of the query. It includes a section for 'STDIN' (Input for the program { Optional }) and an 'Output:' section displaying the results of the SQL query in a table format:

id	name
4	K ADVAITH
5	E HEMANTH

The status bar at the bottom indicates 'Ready'.

PART 5 — RIGHT OUTER JOIN

1. Right Join — Show All Address Records



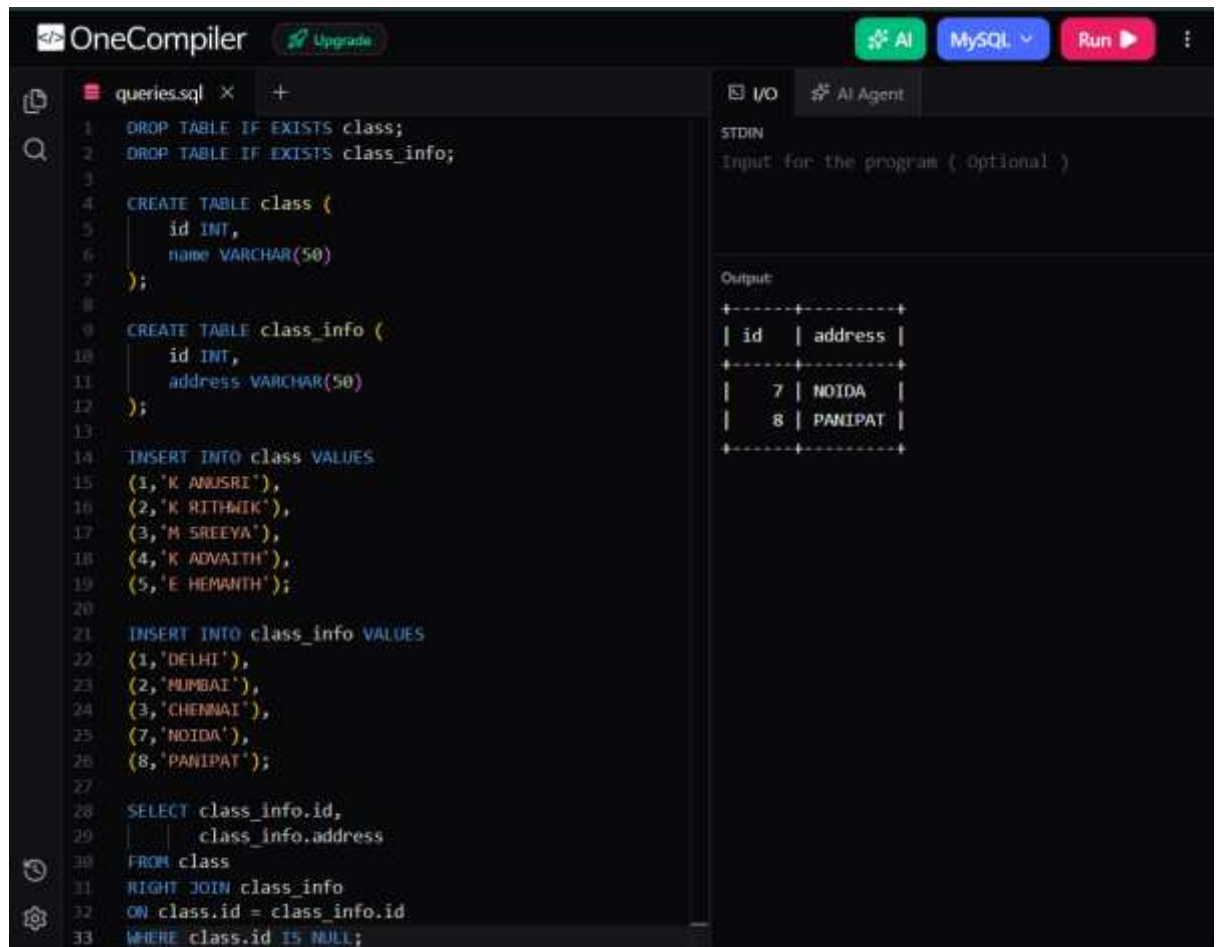
The screenshot shows the OneCompiler IDE interface. The left pane displays a SQL query in a file named `queries.sql`. The query creates two tables, `class` and `class_info`, and performs a right join between them. The right pane shows the output of the query, which is a table with 4 columns: `id`, `name`, `id`, and `address`. The output table contains 6 rows of data, including rows with null values.

```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10    id INT,
11    address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1, 'K ANUSRI'),
16 (2, 'K RITHWIK'),
17 (3, 'M SREEYA'),
18 (4, 'K ADVAITH'),
19 (5, 'E HEMANTH');
20
21 INSERT INTO class_info VALUES
22 (1, 'DELHI'),
23 (2, 'MUMBAI'),
24 (3, 'CHENNAI'),
25 (7, 'NOIDA'),
26 (8, 'PANIPAT');
27
28 SELECT *
29 FROM class
30 RIGHT JOIN class_info
31 ON class.id = class_info.id;
```

Output:

id	name	id	address
1	K ANUSRI	1	DELHI
2	K RITHWIK	2	MUMBAI
3	M SREEYA	3	CHENNAI
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

2. Right Join — Find Addresses Without Students



The screenshot shows the OneCompiler IDE with a MySQL query in a file named `queries.sql`. The query performs a right join between `class` and `class_info` tables to find addresses without students. The output shows two rows: `(7, NOIDA)` and `(8, PANIPAT)`.

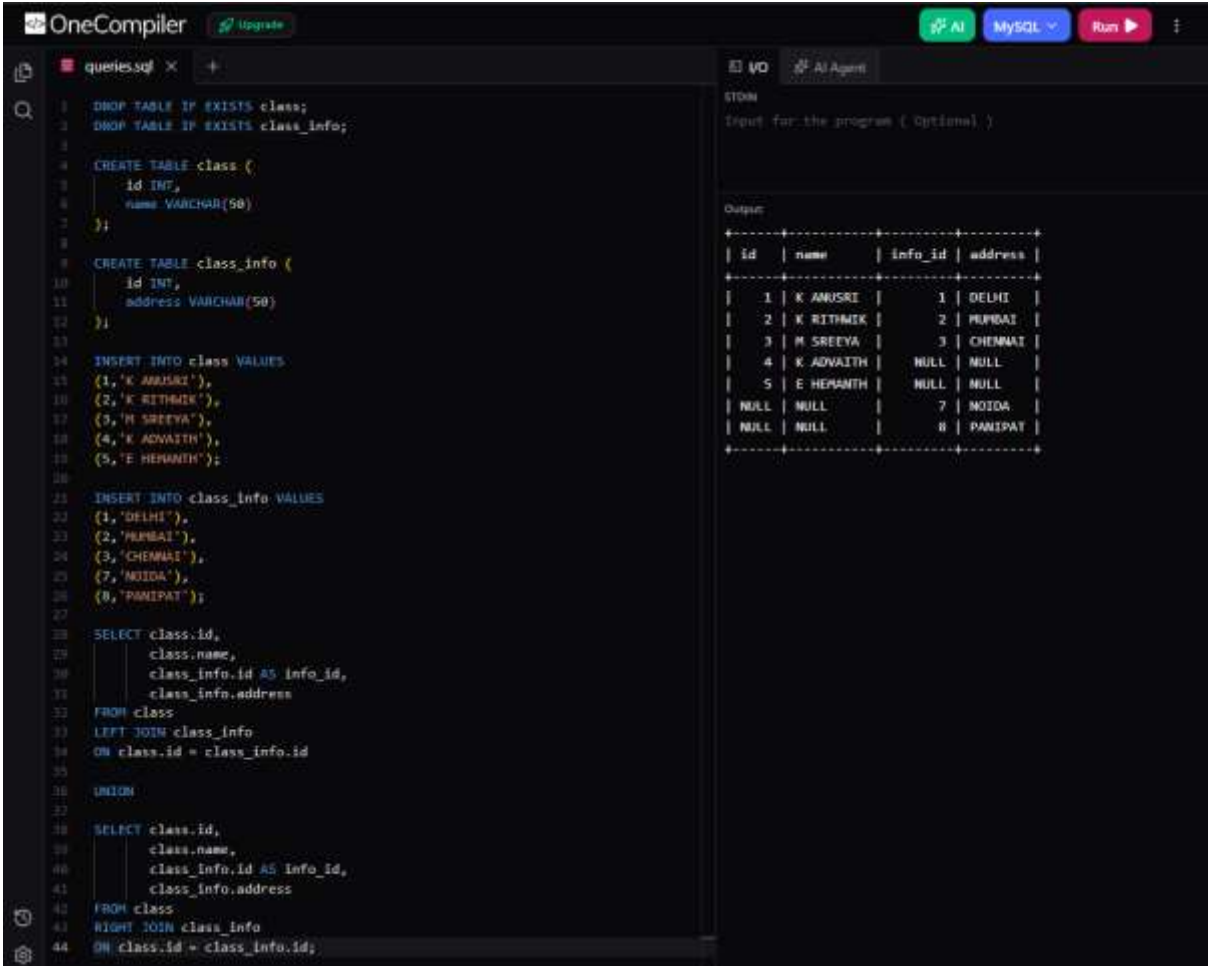
```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10    id INT,
11    address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1, 'K. ANUSRI'),
16 (2, 'K. RITHVIK'),
17 (3, 'M. SREEYA'),
18 (4, 'K. ADVAITH'),
19 (5, 'E. HEMANTH');
20
21 INSERT INTO class_info VALUES
22 (1, 'DELHI'),
23 (2, 'MUMBAI'),
24 (3, 'CHENNAI'),
25 (7, 'NOIDA'),
26 (8, 'PANIPAT');
27
28 SELECT class_info.id,
29        class_info.address
30 FROM class
31 RIGHT JOIN class_info
32 ON class.id = class_info.id
33 WHERE class.id IS NULL;
```

Output:

id	address
7	NOIDA
8	PANIPAT

PART 6 — FULL OUTER JOIN

1. Full Outer Join — Display All Matching and Unmatched Records



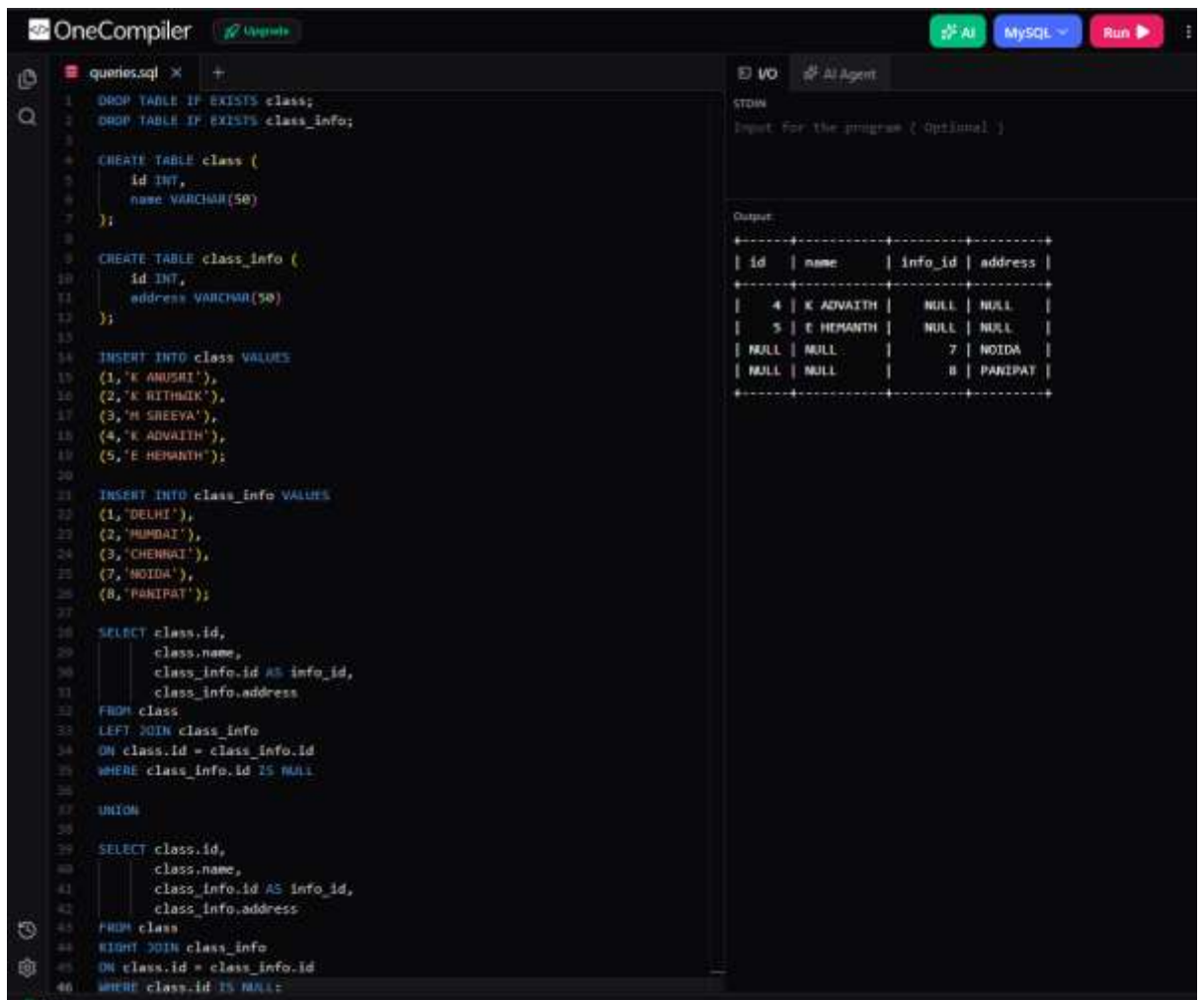
The screenshot shows the OneCompiler interface with a MySQL query editor. The query performs a Full Outer Join between the 'class' and 'class_info' tables. The 'class' table contains student records, and the 'class_info' table contains location records. The query uses a combination of LEFT JOIN, RIGHT JOIN, and UNION to display all records from both tables, including those without matches.

```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5   id INT,
6   name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10  id INT,
11  address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1, 'K AMRUSRI'),
16 (2, 'K RITHMEK'),
17 (3, 'M SREEYA'),
18 (4, 'K ADVAITH'),
19 (5, 'E HERMANTH');
20
21 INSERT INTO class_info VALUES
22 (1, 'DELHI'),
23 (2, 'MUMBAI'),
24 (3, 'CHENNAI'),
25 (7, 'MOIDA'),
26 (8, 'PANIPAT');
27
28 SELECT class.id,
29        class.name,
30        class_info.id AS info_id,
31        class_info.address
32 FROM class
33 LEFT JOIN class_info
34 ON class.id = class_info.id
35
36 UNION
37
38 SELECT class.id,
39        class.name,
40        class_info.id AS info_id,
41        class_info.address
42 FROM class
43 RIGHT JOIN class_info
44 ON class.id = class_info.id;
```

Output:

id	name	info_id	address
1	K AMRUSRI	1	DELHI
2	K RITHMEK	2	MUMBAI
3	M SREEYA	3	CHENNAI
4	K ADVAITH	NULL	NULL
5	E HERMANTH	NULL	NULL
NULL	NULL	7	MOIDA
NULL	NULL	8	PANIPAT

2. Full Outer Join — Display Only Unmatched Records



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script with the following queries:

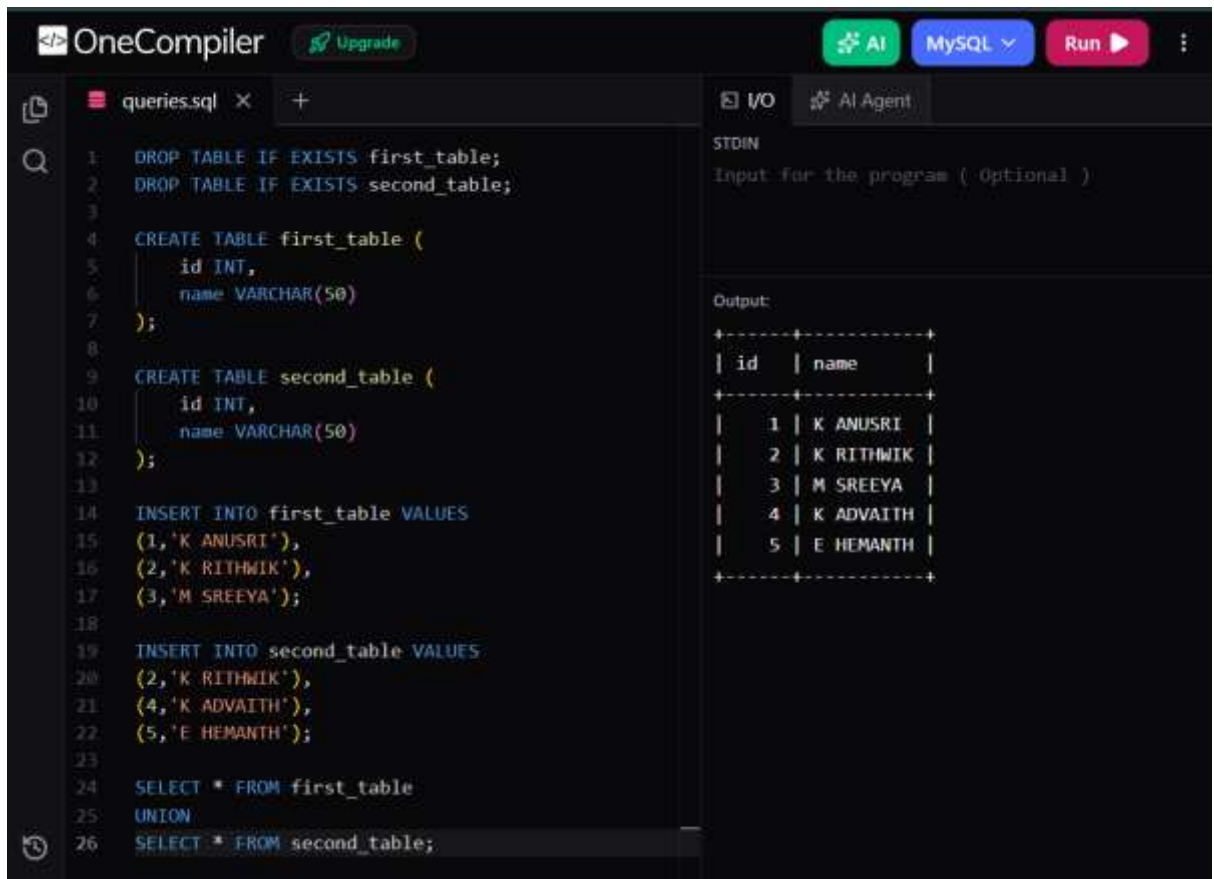
```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5   id INT,
6   name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10  id INT,
11  address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1, 'K ANUSHI'),
16 (2, 'K RITHMIK'),
17 (3, 'M SHREEVA'),
18 (4, 'K ADVAITH'),
19 (5, 'E HEMANTH');
20
21 INSERT INTO class_info VALUES
22 (1, 'DELHI'),
23 (2, 'MUMBAI'),
24 (3, 'CHENNAI'),
25 (7, 'NOIDA'),
26 (8, 'PANIPAT');
27
28 SELECT class.id,
29        class.name,
30        class_info.id AS info_id,
31        class_info.address
32 FROM class
33 LEFT JOIN class_info
34 ON class.id = class_info.id
35 WHERE class_info.id IS NULL;
36
37 UNION
38
39 SELECT class.id,
40        class.name,
41        class_info.id AS info_id,
42        class_info.address
43 FROM class
44 RIGHT JOIN class_info
45 ON class.id = class_info.id
46 WHERE class.id IS NULL;
```

The right pane shows the output of the query, which is a table with 4 columns: id, name, info_id, and address. The output displays only the unmatched records from both tables.

id	name	info_id	address
4	K ADVAITH	NULL	NULL
5	E HEMANTH	NULL	NULL
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

PART 7 - UNION

1. UNION — Combine Student Lists Without Duplicates



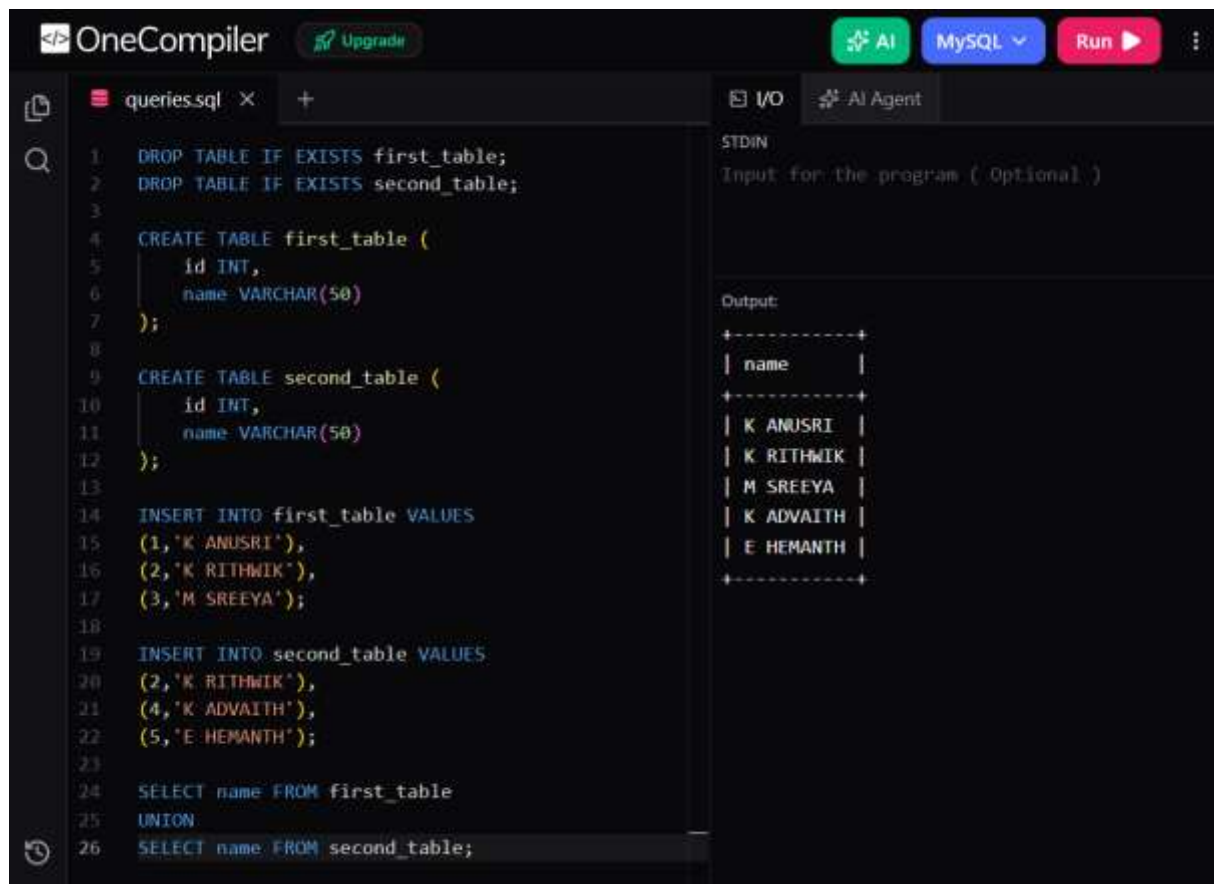
The screenshot shows the OneCompiler MySQL interface. The left pane contains a file named 'queries.sql' with the following SQL code:

```
1 DROP TABLE IF EXISTS first_table;
2 DROP TABLE IF EXISTS second_table;
3
4 CREATE TABLE first_table (
5   id INT,
6   name VARCHAR(50)
7 );
8
9 CREATE TABLE second_table (
10  id INT,
11  name VARCHAR(50)
12 );
13
14 INSERT INTO first_table VALUES
15 (1, 'K ANUSRI'),
16 (2, 'K RITHWIK'),
17 (3, 'M SREEYA');
18
19 INSERT INTO second_table VALUES
20 (2, 'K RITHWIK'),
21 (4, 'K ADVAITH'),
22 (5, 'E HEMANTH');
23
24 SELECT * FROM first_table
25 UNION
26 SELECT * FROM second_table;
```

The right pane shows the output of the queries. It includes a 'STDOUT' section with the text 'Input for the program (Optional)' and an 'Output:' section displaying a table of results:

id	name
1	K ANUSRI
2	K RITHWIK
3	M SREEYA
4	K ADVAITH
5	E HEMANTH

2. UNION — Generate Master Student Name List



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script in a file named 'queries.sql'. The script defines two tables, 'first_table' and 'second_table', each with an 'id' integer and a 'name' varchar(50). It inserts three rows into 'first_table' and two rows into 'second_table'. Finally, it uses a UNION query to select names from both tables. The right pane shows the output of the query, which is a list of names: K ANUSRI, K RITHWIK, M SREEYA, K ADVAITH, and E HEMANTH.

```
1 DROP TABLE IF EXISTS first_table;
2 DROP TABLE IF EXISTS second_table;
3
4 CREATE TABLE first_table (
5   id INT,
6   name VARCHAR(50)
7 );
8
9 CREATE TABLE second_table (
10  id INT,
11  name VARCHAR(50)
12 );
13
14 INSERT INTO first_table VALUES
15 (1, 'K ANUSRI'),
16 (2, 'K RITHWIK'),
17 (3, 'M SREEYA');
18
19 INSERT INTO second_table VALUES
20 (2, 'K RITHWIK'),
21 (4, 'K ADVAITH'),
22 (5, 'E HEMANTH');
23
24 SELECT name FROM first_table
25 UNION
26 SELECT name FROM second_table;
```

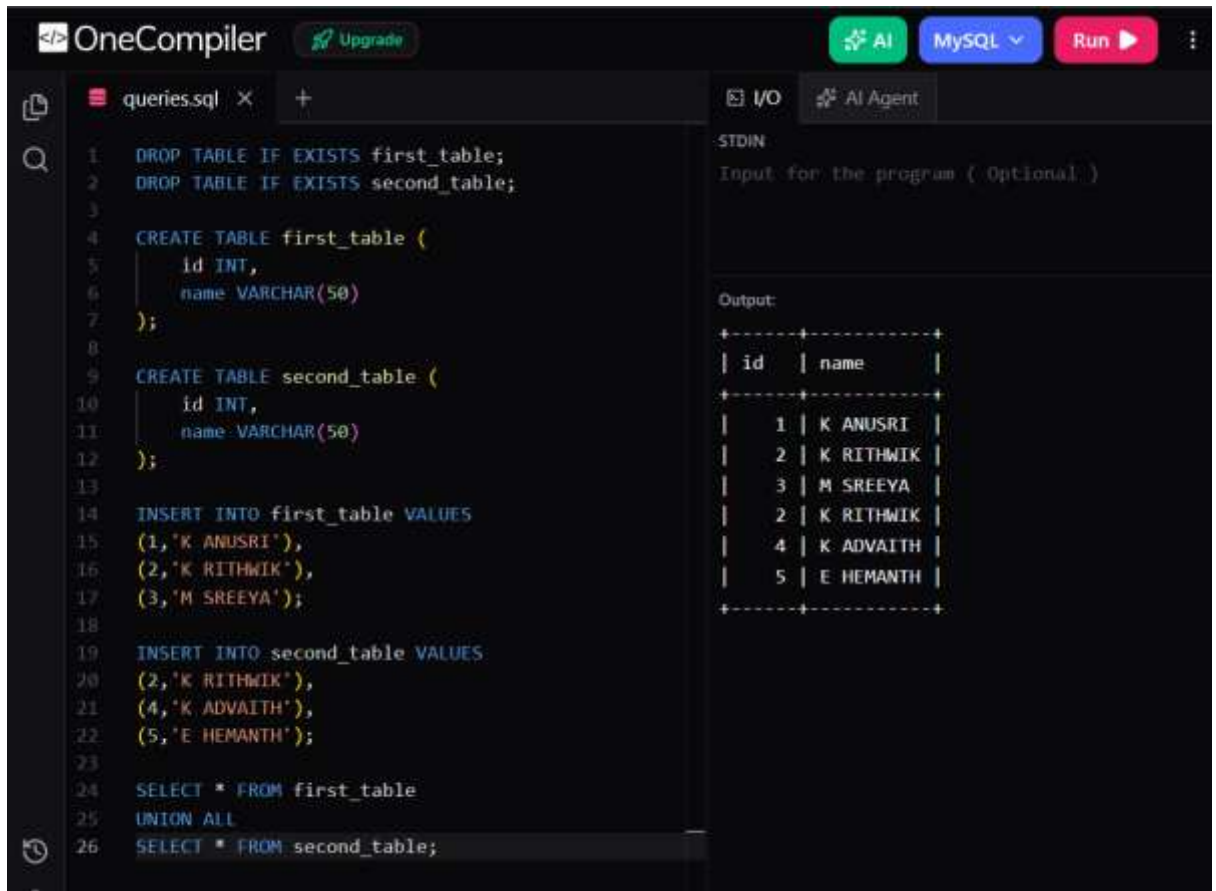
STDIN
Input for the program (Optional)

Output:

name
K ANUSRI
K RITHWIK
M SREEYA
K ADVAITH
E HEMANTH

PART 8 — UNION ALL

1. UNION ALL — Include Duplicate Records



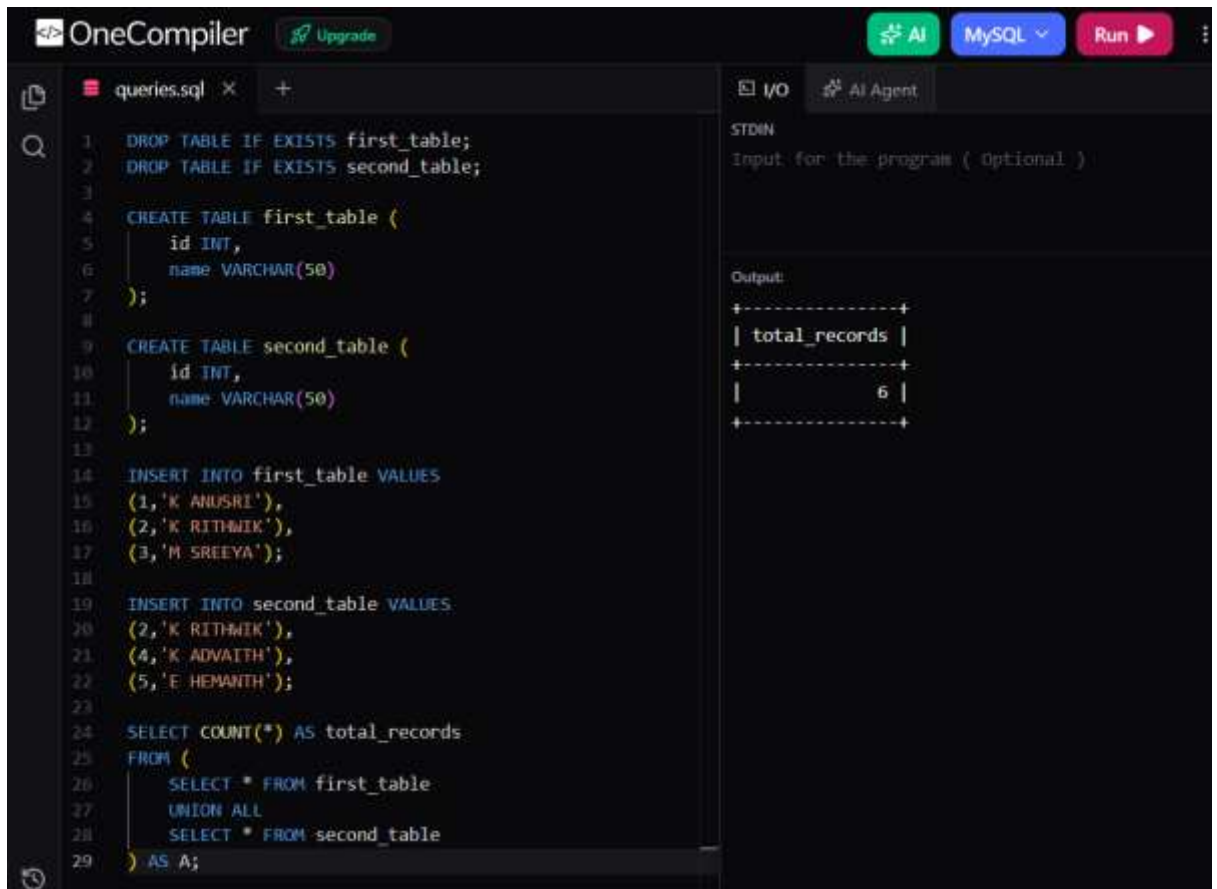
The screenshot shows the OneCompiler MySQL interface. The SQL code in the editor is as follows:

```
1 DROP TABLE IF EXISTS first_table;
2 DROP TABLE IF EXISTS second_table;
3
4 CREATE TABLE first_table (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE second_table (
10    id INT,
11    name VARCHAR(50)
12 );
13
14 INSERT INTO first_table VALUES
15 (1,'K ANUSRI'),
16 (2,'K RITHWIK'),
17 (3,'M SREEYA');
18
19 INSERT INTO second_table VALUES
20 (2,'K RITHWIK'),
21 (4,'K ADVAITH'),
22 (5,'E HEMANTH');
23
24 SELECT * FROM first_table
25 UNION ALL
26 SELECT * FROM second_table;
```

The output section displays the result of the query, showing a table with 5 rows and 2 columns (id, name). The output is as follows:

id	name
1	K ANUSRI
2	K RITHWIK
3	M SREEYA
2	K RITHWIK
4	K ADVAITH
5	E HEMANTH

2.UNION ALL — Count Total Records



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL query in a file named 'queries.sql'. The query creates two tables, 'first_table' and 'second_table', each with columns 'id' (INT) and 'name' (VARCHAR(50)). 'first_table' is populated with three records, and 'second_table' is populated with three records. The final query uses 'UNION ALL' to combine records from both tables and then uses 'COUNT(*)' to count the total number of records, aliasing the result as 'total_records'.

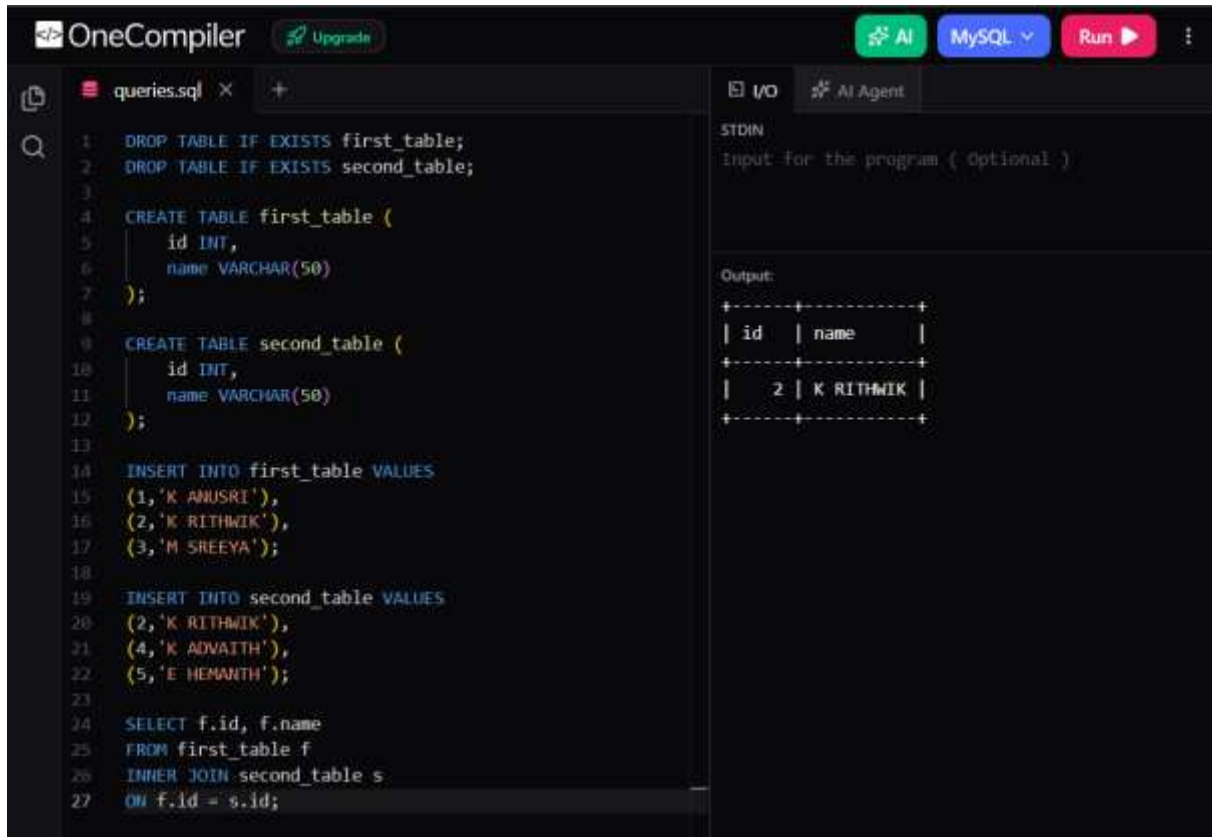
```
1 DROP TABLE IF EXISTS first_table;
2 DROP TABLE IF EXISTS second_table;
3
4 CREATE TABLE first_table (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE second_table (
10    id INT,
11    name VARCHAR(50)
12 );
13
14 INSERT INTO first_table VALUES
15 (1,'K ANUSRI'),
16 (2,'K RITHVIK'),
17 (3,'M SREEYA');
18
19 INSERT INTO second_table VALUES
20 (2,'K RITHVIK'),
21 (4,'K ADVAITH'),
22 (5,'E HEMANTH');
23
24 SELECT COUNT(*) AS total_records
25 FROM (
26     SELECT * FROM first_table
27     UNION ALL
28     SELECT * FROM second_table
29 ) AS A;
```

The right pane shows the output of the query. It displays a table with one column, 'total_records', and one row with the value '6'. The output is formatted with a header row and a data row, separated by a horizontal line.

total_records
6

PART 9 — INTERSECT

1. INTERSECT — Find Students Present in Both Tables

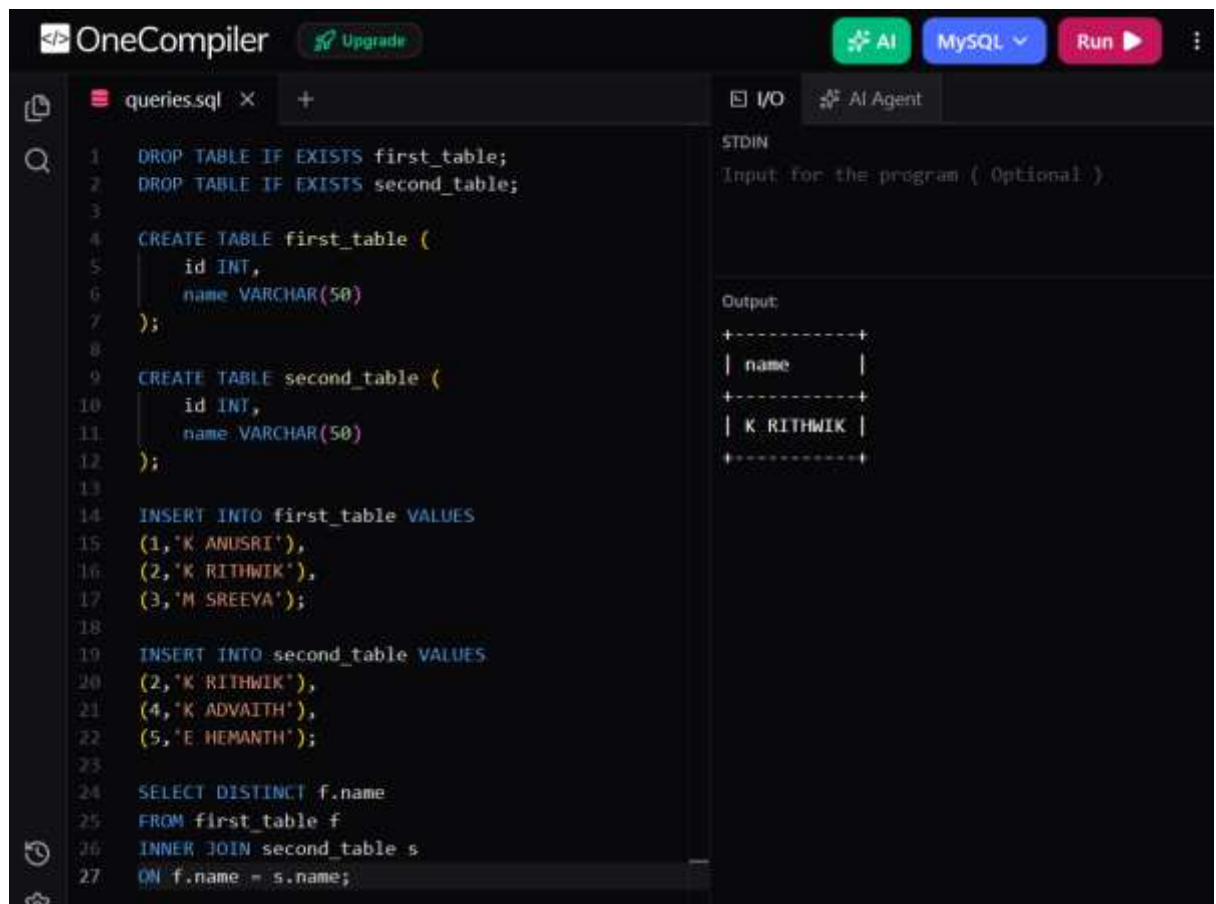


```
1 DROP TABLE IF EXISTS first_table;
2 DROP TABLE IF EXISTS second_table;
3
4 CREATE TABLE first_table (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE second_table (
10    id INT,
11    name VARCHAR(50)
12 );
13
14 INSERT INTO first_table VALUES
15 (1,'K ANUSRI'),
16 (2,'K RITHWIK'),
17 (3,'M SREEYA');
18
19 INSERT INTO second_table VALUES
20 (2,'K RITHWIK'),
21 (4,'K ADVAITH'),
22 (5,'E HEMANTH');
23
24 SELECT f.id, f.name
25 FROM first_table f
26 INNER JOIN second_table s
27 ON f.id = s.id;
```

Output:

id	name
2	K RITHWIK

2. INTERSECT — Find Common Student Names



The screenshot shows the OneCompiler MySQL interface. The code editor on the left contains the following SQL queries:

```
1 DROP TABLE IF EXISTS first_table;
2 DROP TABLE IF EXISTS second_table;
3
4 CREATE TABLE first_table (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE second_table (
10    id INT,
11    name VARCHAR(50)
12 );
13
14 INSERT INTO first_table VALUES
15 (1,'K ANUSRI'),
16 (2,'K RITHWIK'),
17 (3,'M SREEYA');
18
19 INSERT INTO second_table VALUES
20 (2,'K RITHWIK'),
21 (4,'K ADVAITH'),
22 (5,'E HEMANTH');
23
24 SELECT DISTINCT f.name
25 FROM first_table f
26 INNER JOIN second_table s
27 ON f.name = s.name;
```

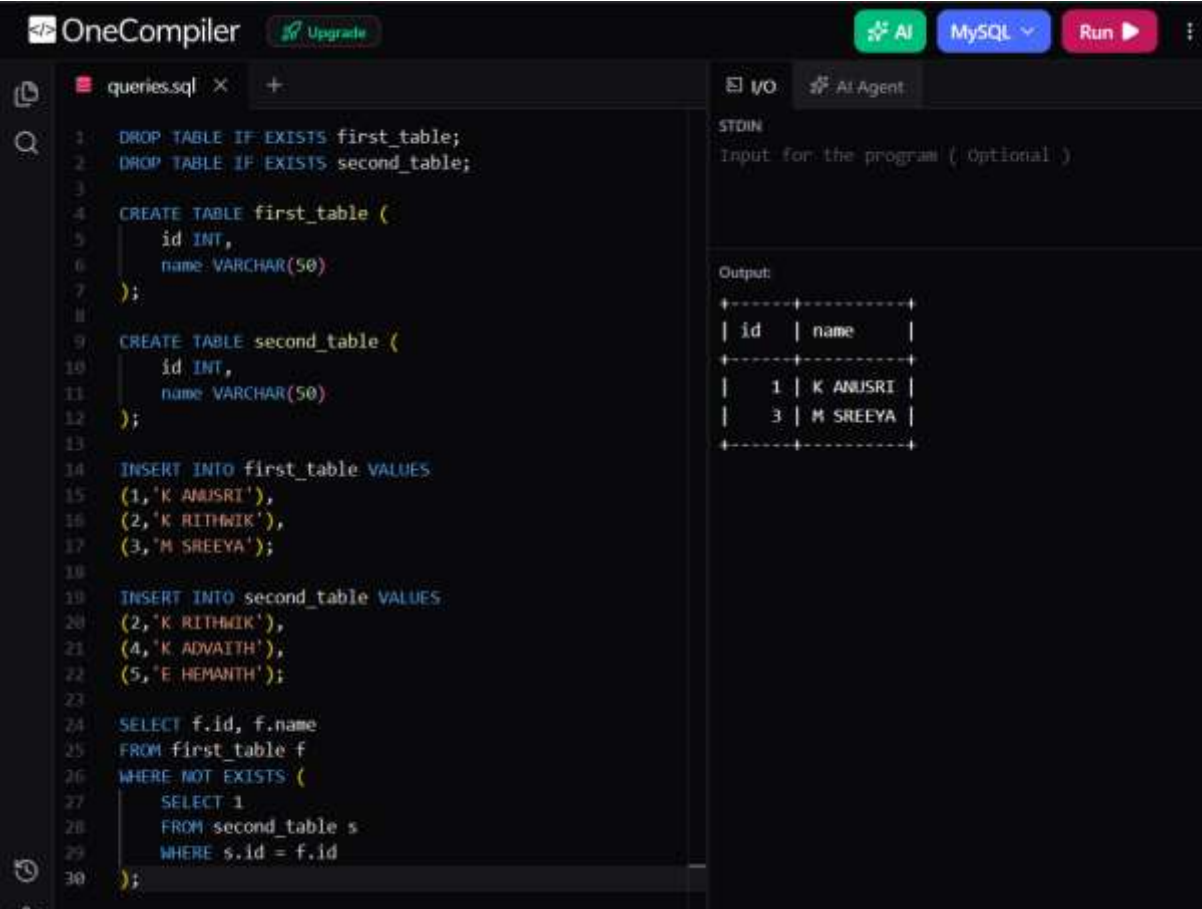
The right panel shows the output of the query. The output is displayed in a table format:

name
K RITHWIK

The output shows that the only common name between the two tables is 'K RITHWIK'.

PART 10 — MINUS

1. MINUS — Find Students Only in First Table



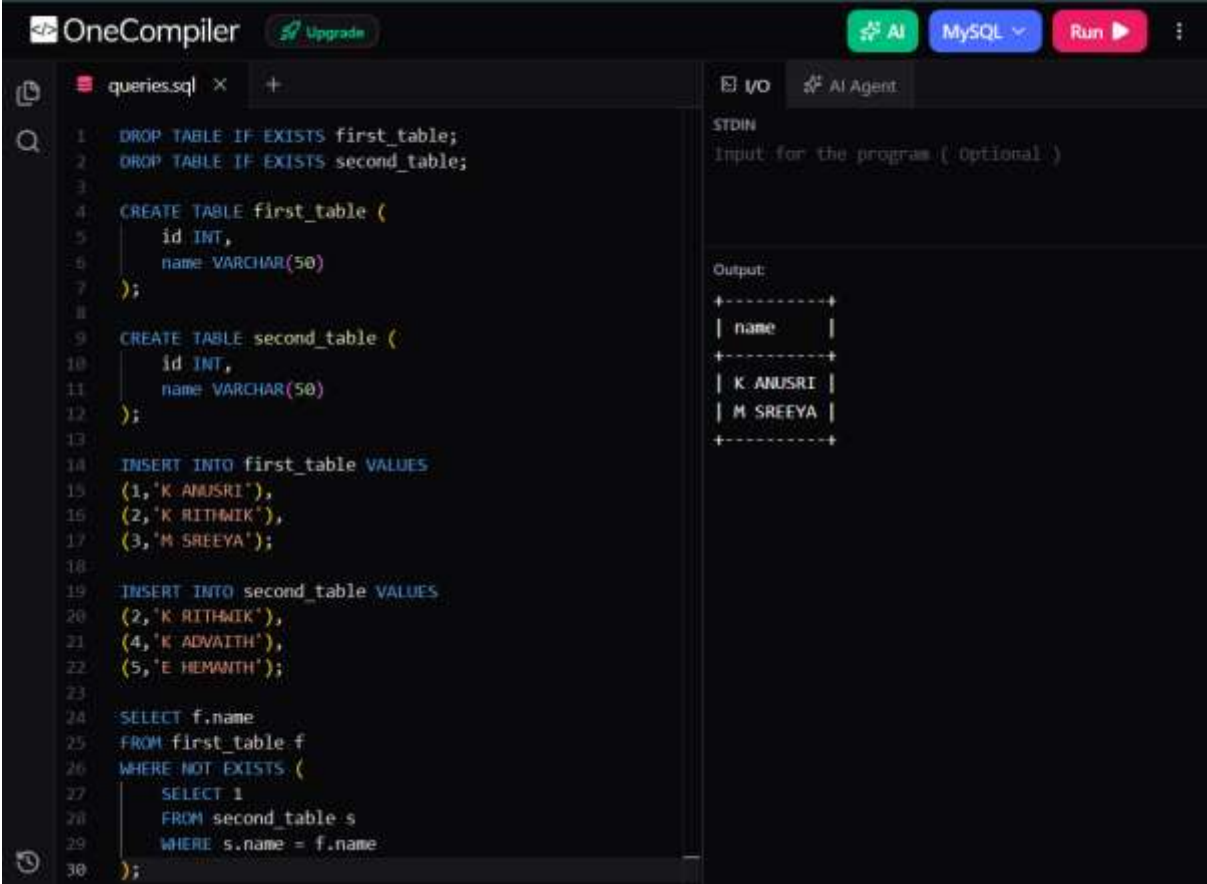
The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script in a file named 'queries.sql'. The script defines two tables, 'first_table' and 'second_table', each with columns 'id' (INT) and 'name' (VARCHAR(50)). 'first_table' contains three rows: (1, 'K ANJUSRI'), (2, 'K RITHVIK'), and (3, 'M SREEYA'). 'second_table' contains three rows: (2, 'K RITHVIK'), (4, 'K ADVAITH'), and (5, 'E HEMANTH'). The final query uses the MINUS operator to find students in 'first_table' who are not in 'second_table', returning two rows: (1, 'K ANJUSRI') and (3, 'M SREEYA'). The right pane shows the output of the query in a table format.

```
1 DROP TABLE IF EXISTS first_table;
2 DROP TABLE IF EXISTS second_table;
3
4 CREATE TABLE first_table (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE second_table (
10    id INT,
11    name VARCHAR(50)
12 );
13
14 INSERT INTO first_table VALUES
15 (1, 'K ANJUSRI'),
16 (2, 'K RITHVIK'),
17 (3, 'M SREEYA');
18
19 INSERT INTO second_table VALUES
20 (2, 'K RITHVIK'),
21 (4, 'K ADVAITH'),
22 (5, 'E HEMANTH');
23
24 SELECT f.id, f.name
25 FROM first_table f
26 WHERE NOT EXISTS (
27     SELECT 1
28     FROM second_table s
29     WHERE s.id = f.id
30 );
```

Output:

id	name
1	K ANJUSRI
3	M SREEYA

2. MINUS — Find Names Only in First Table



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script named 'queries.sql' with the following code:

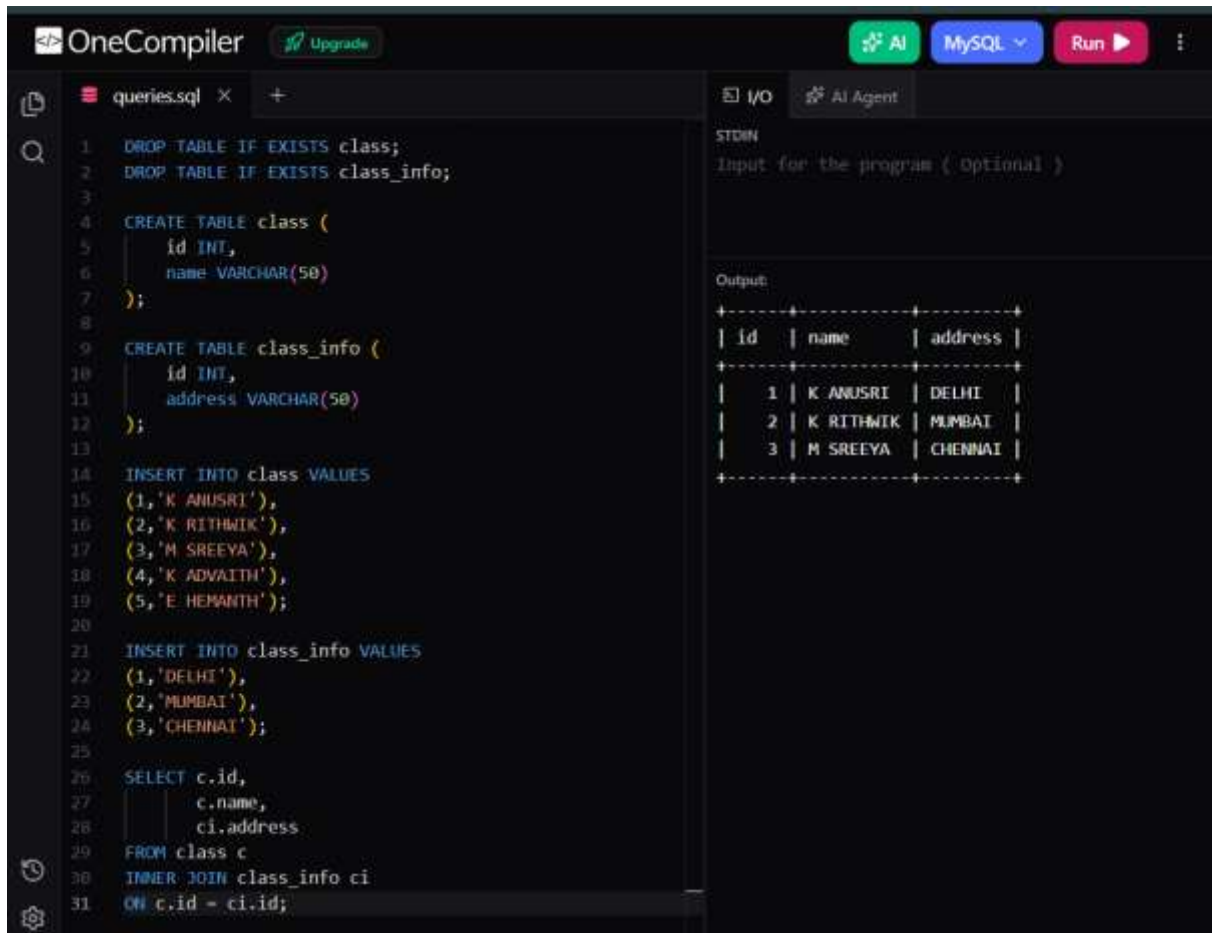
```
1 DROP TABLE IF EXISTS first_table;
2 DROP TABLE IF EXISTS second_table;
3
4 CREATE TABLE first_table (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE second_table (
10    id INT,
11    name VARCHAR(50)
12 );
13
14 INSERT INTO first_table VALUES
15 (1,'K ANUSRI'),
16 (2,'K RITHWIK'),
17 (3,'M SREEYA');
18
19 INSERT INTO second_table VALUES
20 (2,'K RITHWIK'),
21 (4,'K ADVAITH'),
22 (5,'E HEMANTH');
23
24 SELECT f.name
25 FROM first_table f
26 WHERE NOT EXISTS (
27     SELECT 1
28     FROM second_table s
29     WHERE s.name = f.name
30 );
```

The right pane shows the output of the query. It includes a section for 'STDIN' with the text 'Input for the program (Optional)' and an 'Output:' section displaying the result of the MINUS query:

name
K ANUSRI
M SREEYA

ADVANCED INTERVIEW QUESTIONS

1. Display Students with Matching Addresses



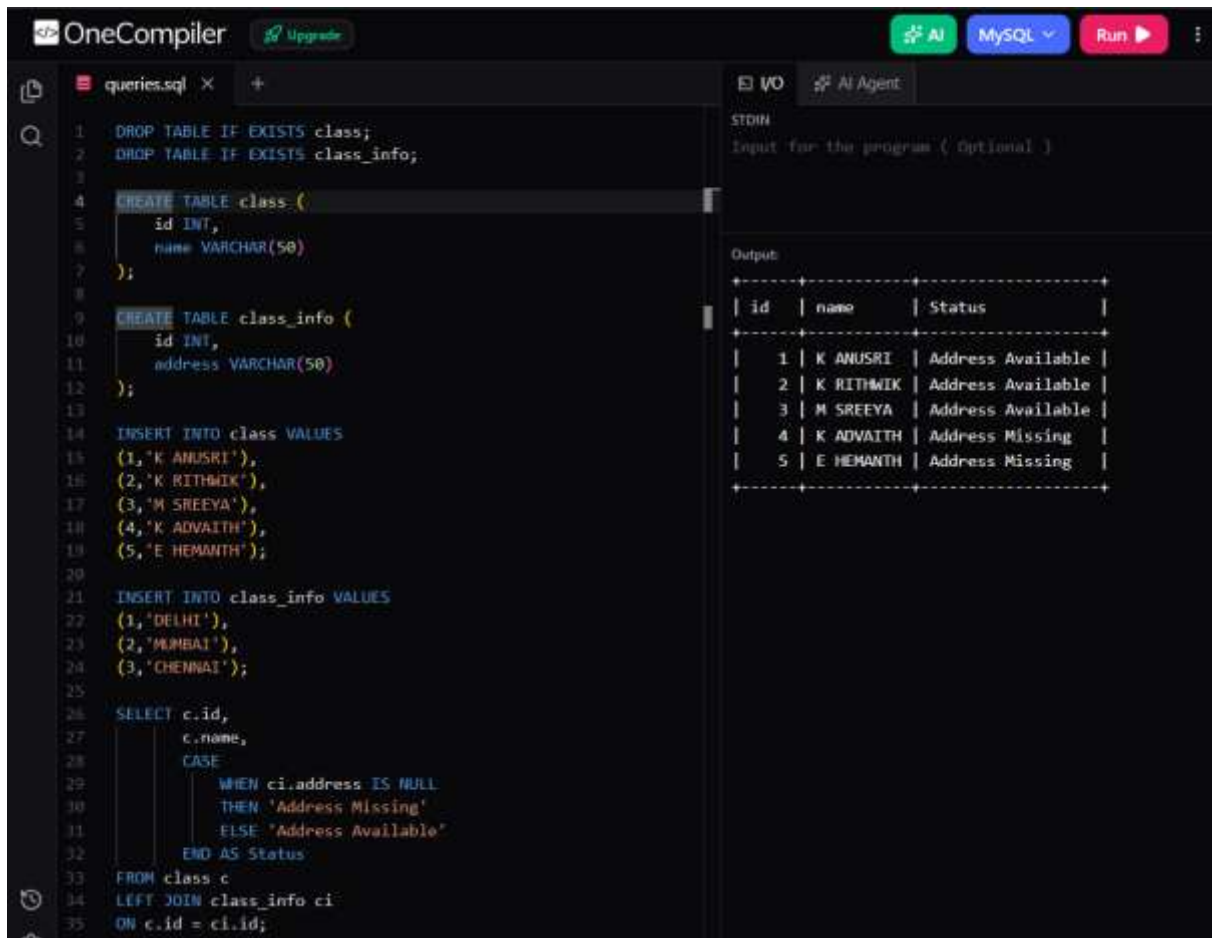
The screenshot shows the OneCompiler IDE interface. The left pane displays a SQL query in a file named 'queries.sql'. The query creates two tables, 'class' and 'class_info', inserts data into them, and then performs an inner join to display students with matching addresses. The right pane shows the output of the query, which is a table with three columns: 'id', 'name', and 'address'. The output table contains three rows of data.

```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10    id INT,
11    address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1,'K ANUSRI'),
16 (2,'K RITHWIK'),
17 (3,'M SREEYA'),
18 (4,'K ADVAITH'),
19 (5,'E HEMANTH');
20
21 INSERT INTO class_info VALUES
22 (1,'DELHI'),
23 (2,'MUMBAI'),
24 (3,'CHENNAI');
25
26 SELECT c.id,
27        c.name,
28        ci.address
29 FROM class c
30 INNER JOIN class_info ci
31 ON c.id = ci.id;
```

Output:

id	name	address
1	K ANUSRI	DELHI
2	K RITHWIK	MUMBAI
3	M SREEYA	CHENNAI

2. Address Availability Report



The screenshot shows the OneCompiler MySQL interface. The left pane contains a file named `queries.sql` with the following SQL code:

```
1 DROP TABLE IF EXISTS class;
2 DROP TABLE IF EXISTS class_info;
3
4 CREATE TABLE class (
5     id INT,
6     name VARCHAR(50)
7 );
8
9 CREATE TABLE class_info (
10     id INT,
11     address VARCHAR(50)
12 );
13
14 INSERT INTO class VALUES
15 (1, 'K ANUSRI'),
16 (2, 'K RITHVIK'),
17 (3, 'M SREEYA'),
18 (4, 'K ADVAITH'),
19 (5, 'E HEMANTH');
20
21 INSERT INTO class_info VALUES
22 (1, 'DELHI'),
23 (2, 'MUMBAI'),
24 (3, 'CHENNAI');
25
26 SELECT c.id,
27        c.name,
28        CASE
29            WHEN ci.address IS NULL
30            THEN 'Address Missing'
31            ELSE 'Address Available'
32        END AS Status
33 FROM class c
34 LEFT JOIN class_info ci
35 ON c.id = ci.id;
```

The right pane shows the output of the queries. It includes a section for `STDIN` (Input for the program (Optional)) and an `Output` section displaying the results of the final `SELECT` query in a table format:

id	name	Status
1	K ANUSRI	Address Available
2	K RITHVIK	Address Available
3	M SREEYA	Address Available
4	K ADVAITH	Address Missing
5	E HEMANTH	Address Missing